

Manual for IQA Subjective Test

For internal reference only. Do not distribute.

Contents

1	Overview	2
2	Code Structure	3
2.1	Data Models	4
2.2	Sampling Strategies	4
3	Command-Line Reference	5
3.1	manage.py — Django Management	5
3.1.1	Initial Setup	5
3.1.2	import_study — Import or Append Stimuli	5
3.1.3	export_responses — Export Responses to CSV	7
3.2	scripts/generate_dev_test.py — Generate Dev Test Dataset	8
3.2.1	createsuperuser — Create Admin Account	9
3.2.2	runserver — Start Development Server	10
4	Researcher's Guide	11
4.1	Step 1: Installation	11
4.2	Step 2: Prepare Images	11
4.3	Step 3: Create a Study	11
4.3.1	Option A: JSON import (recommended for large studies)	11
4.3.2	Option B: Django admin panel	11
4.4	Step 4: Create Subject Accounts	12
4.4.1	Option A: Bulk creation via the web interface	12
4.4.2	Option B: Django admin	12
4.5	Step 5: Run the Test	12
4.6	Step 6: Monitor and Export Results	12
4.6.1	Web viewer	12
4.6.2	CSV export (web)	12
4.6.3	CSV export (command line)	13
4.7	Study Configuration Reference	13
4.7.1	Image Sizing Modes	13
4.7.2	Zoom Overlay	14
5	Subject's Guide	15
5.1	Logging In	15
5.2	Home Page	15
5.3	MOS Evaluation (Single Image Rating)	15
5.4	2AFC Evaluation (Pair Comparison)	15
5.5	General Tips	16
5.6	Logging Out	16
5.7	Additional Instructions for Subject Running the Server on their Own Machine	16
6	URL Reference	17

1 Overview

This tool is a Django-based web application for conducting **Image Quality Assessment (IQA) subjective tests**. It supports two evaluation modes:

1. **MOS (Mean Opinion Score)** — A single test image is shown (with an optional reference image displayed alongside). The subject rates the image quality on a configurable integer scale (default 1–5).
2. **2AFC (Two-Alternative Forced Choice)** — A pair of test images is shown side by side (each with an optional reference image). The subject selects which image has better quality.

The system handles user authentication, stimulus sampling, progress tracking, response storage, and CSV export. All configuration is done through the Django admin interface or via a JSON import command.

2 Code Structure

The project lives in the *iqa_subjective_test/* directory.

```
iqa_subjective_test/
├── manage.py           Django entry point
├── requirements.txt    Python dependencies
├── images/            Image storage directory (MEDIA_ROOT-relative)
│   └── .gitkeep
├── iqa_site/          Django project package
│   ├── __init__.py
│   ├── settings.py    Project settings (DB, media, auth URLs)
│   ├── urls.py        Root URL conf (admin + iqa app)
│   ├── wsgi.py
│   └── asgi.py
├── iqa/               Main application
│   ├── __init__.py
│   ├── apps.py        App configuration
│   ├── models.py      Data models (6 models)
│   ├── views.py       View functions (11 views)
│   ├── urls.py        URL patterns (app_name = 'iqa')
│   ├── admin.py       Admin registrations + inlines
│   ├── forms.py       Bulk user creation form
│   └── samplers.py    Stimulus sampling strategies
│   ├── templatetags/
│   │   ├── __init__.py
│   │   └── iqa_extras.py    Template filter: get_item
│   ├── management/
│   │   └── commands/
│   │       ├── import_study.py    JSON study import command
│   │       └── export_responses.py CSV response export command
│   ├── migrations/          Django migrations (auto-generated)
│   ├── static/iqa/
│   │   ├── css/
│   │   │   └── evaluation.css    Evaluation page styles
│   │   └── js/
│   │       └── zoom.js           Hover-to-zoom overlay logic
│   └── templates/
│       ├── base.html          Base template (shared layout)
│       └── iqa/
│           ├── login.html      Login page
│           ├── home.html       Study listing / dashboard
│           ├── mos_evaluation.html    MOS rating interface
│           ├── pair_evaluation.html    2AFC choice interface
│           └── next_stimulus_redirect.html    Auto-redirect after submit
```

- |— `study_done.html` Completion message
- |— `view_responses.html` Staff response viewer
- |— `bulk_create_users.html` Bulk user creation form
- |— `user_creation_results.html` Created credentials display

2.1 Data Models

Model	Purpose
Image	Stores an image file (<i>images/</i> upload directory) and an optional display name.
Study	Top-level configuration: name, mode (MOS or 2AFC), prompt text, scale range and labels (MOS only), sampling strategy, and active flag.
MOSStimulus	Links a test image to a Study, with an optional reference image. Has an order field for presentation sequencing.
PairStimulus	Links image_a and image_b to a Study, each with optional reference_a / reference_b. Has an order field.
MOSResponse	Stores a subject's integer score for a MOS stimulus. Unique per (stimulus, user) pair.
PairResponse	Stores a subject's choice (A or B) for a pair stimulus. Unique per (stimulus, user) pair.

2.2 Sampling Strategies

The *samplers.py* module provides three strategies, selected per-study via the `sampler` field:

- **sequential** — Presents stimuli in ascending (`order`, `id`). Deterministic ordering.
- **random** — Picks a random unevaluated stimulus each time.
- **least_evaluated** — Picks the stimulus with the fewest total responses across all users, breaking ties by (`order`, `id`). Useful for balancing response counts when subjects may not complete all stimuli.

3 Command-Line Reference

All commands are run from inside the *iqu_subjective_test/* directory.

3.1 manage.py — Django Management

3.1.1 Initial Setup

```
# Create database tables
python manage.py makemigrations iqa
python manage.py migrate
```

```
# Create an administrator account
python manage.py createsuperuser
```

```
# Start the development server
python manage.py runserver [port]
```

The default port is 8000. The application is then accessible at:

- Subject interface: <http://localhost:8000/iqa/>
- Admin interface: <http://localhost:8000/admin/>

3.1.2 import_study — Import or Append Stimuli

Synopsis:

Command:

```
python manage.py import_study [--activate] [--append-to STUDY_ID] <json_file>
```

Description:

Reads a JSON file describing a study and its stimuli, creates the corresponding Study, Image, and stimulus objects in the database. Image paths in the JSON are relative to **MEDIA_ROOT** (which defaults to the *iqu_subjective_test/* directory).

When `--append-to` is given, no new study is created. Instead, the `stimuli` list from the JSON is appended to the existing study identified by `STUDY_ID`. The new stimuli are assigned order values starting after the highest existing order in that study. All other fields in the JSON (`name`, `prompt`, `scale_*`, `sampler`) are ignored in append mode. If the JSON contains a `mode` field, it must match the existing study's mode.

Options:

<code>json_file</code>	Path to the JSON file to import (required positional argument).
<code>--activate</code>	If given, the study is marked <code>is_active=True</code> immediately so subjects can see it. Ignored when appending.
<code>--append-to STUDY_ID</code>	Append the stimuli from the JSON file to an existing study (specified by its numeric ID) instead of creating a new one.

JSON format for MOS:

```
{
  "name": "My MOS Study",
```

```

"mode": "MOS",
"prompt": "Rate the quality of the image.",
"scale_min": 1,
"scale_max": 5,
"scale_min_label": "Bad",
"scale_max_label": "Excellent",
"sampler": "sequential",
"image_sizing": "scaled",
"image_scale_factor": 0.5,
"zoom_enabled": true,
"zoom_factor": 3.0,
"stimuli": [
  {
    "image": "images/distorted_001.png",
    "reference": "images/reference_001.png"
  },
  {
    "image": "images/distorted_002.png"
  }
]
}

```

Each stimulus entry has a required image key and an optional reference key. The scale_*, sampler, image_sizing, image_scale_factor, zoom_enabled, and zoom_factor fields are optional and default to a 1–5 scale, sequential sampling, fit-to-screen sizing, and zoom disabled.

JSON format for 2AFC:

```

{
  "name": "My 2AFC Study",
  "mode": "2AFC",
  "prompt": "Which image has better quality?",
  "sampler": "random",
  "stimuli": [
    {
      "image_a": "images/method_a_001.png",
      "image_b": "images/method_b_001.png",
      "reference_a": "images/ref_001.png",
      "reference_b": "images/ref_001.png"
    },
    {
      "image_a": "images/method_a_002.png",
      "image_b": "images/method_b_002.png"
    }
  ]
}

```

Each stimulus entry requires image_a and image_b. The reference_a and reference_b fields are optional and independent—you can provide a reference for one image but not the other, or the same reference for both.

Examples:

Create a new study and activate it:

Command:

```
python manage.py import_study studies/blur_mos.json --activate
```

Append additional stimuli to study with id=3:

Command:

```
python manage.py import_study extra_images.json --append-to 3
```

3.1.3 export_responses — Export Responses to CSV

Synopsis:

Command:

```
python manage.py export_responses [-s ID_OR_NAME] [-o FILE]
```

Description:

Exports subjective-test responses to CSV. By default all studies are included (they must share the same mode); use `--study` to restrict to a single study. Output goes to stdout unless `--output` is given.

For MOS studies the CSV columns are:

study	stimulus_order	image	reference	user	score	timestamp
-------	----------------	-------	-----------	------	-------	-----------

For 2AFC studies the CSV columns are:

study	stimulus_order	image_a	image_b	reference	reference_index	user	choice	timestamp
-------	----------------	---------	---------	-----------	-----------------	------	--------	-----------

Options:

<code>-s, --study ID_OR_NAME</code>	Export only the specified study. Accepts a numeric ID or the exact study name.
<code>-o, --output FILE</code>	Write CSV to FILE instead of stdout.

Examples:

Export a single study by ID to a file:

Command:

```
python manage.py export_responses --study 3 -o results.csv
```

Export a study by name, piping to another tool:

Command:

```
python manage.py export_responses -s "Blur MOS" | head
```

Export all studies (stdout):

Command:

```
python manage.py export_responses
```

3.2 scripts/generate_dev_test.py — Generate Dev Test Dataset

Synopsis:

Command:

```
python scripts/generate_dev_test.py [options] <live_dir>
```

Description:

Generates a development test image set from the LIVE IQA dataset. The script randomly samples distorted images from a CSV file list, center-crops each distorted image and its corresponding reference to 512 × 512 pixels, saves them as PNG files under the output directory, and produces a JSON file ready for import_study.

Two modes are supported:

- **MOS** (default): each stimulus is a single distorted image with its reference.
- **2AFC**: stimuli are pairs of distorted images. By default all pairs share the same reference scene. The `--cross-ref FRAC` option controls the fraction of pairs drawn from *different* reference scenes (each side gets its own reference image). For example, `--cross-ref 0.3` produces 70% same-scene pairs and 30% cross-scene pairs.

The LIVE dataset root directory must contain the standard subdirectory layout:

```
<live_dir>/
├─ refimgs/      reference BMPs
├─ jp2k/         JPEG 2000 distortions
├─ jpeg/         JPEG distortions
├─ wn/           white noise
├─ gblur/        Gaussian blur
└─ fastfading/   fast-fading (Rayleigh) distortions
```

Positional argument:

live_dir	Root directory of the LIVE dataset (must contain a <i>refimgs/</i> subdirectory).
----------	---

Options:

--mode {MOS,2AFC}	Study mode. Default: MOS.
--cross-ref FRAC	2AFC only. Fraction (0.0–1.0) of pairs drawn from different reference scenes. Default 0.0 (all same-scene). Setting 1.0 makes all pairs cross-scene.
--csv PATH	Path to the filelist CSV. Default: <i>tmp/filelist.csv</i> .
--outdir PATH	Output directory for cropped PNG images. Default: <i>images/live_dev/</i> .
--json PATH	Output path for the generated study JSON. Default: <i>scripts/live_dev_mos.json</i> for MOS, <i>scripts/live_dev_2afc.json</i> for 2AFC.

-n N	Number of stimuli to sample (individual images for MOS, pairs for 2AFC). Default: 50.
--seed SEED	Random seed for reproducible sampling. Default: 42.

Outputs:

- Cropped 512×512 PNG images in `<outdir>/<distortion_type>/` and `<outdir>/refimgs/`.
- A study JSON file at `<json>` with paths relative to `MEDIA_ROOT`.

Example workflow (MOS):

Command:

```
# 1. Generate the dev test images and JSON
python scripts/generate_dev_test.py /data/LIVE

# 2. Import the generated study
python manage.py import_study scripts/live_dev_mos.json --activate

# 3. Start the server and evaluate
python manage.py runserver
```

Example workflow (2AFC, all same-scene):

Command:

```
python scripts/generate_dev_test.py /data/LIVE --mode 2AFC
python manage.py import_study scripts/live_dev_2afc.json --activate
```

Example workflow (2AFC, mixed 70/30):

Command:

```
python scripts/generate_dev_test.py /data/LIVE --mode 2AFC --cross-ref 0.3
python manage.py import_study scripts/live_dev_2afc.json --activate
```

Example workflow (2AFC, all cross-scene):

Command:

```
python scripts/generate_dev_test.py /data/LIVE --mode 2AFC --cross-ref 1.0
python manage.py import_study scripts/live_dev_2afc.json --activate
```

With custom paths:

Command:

```
python scripts/generate_dev_test.py /data/LIVE \
  --mode 2AFC \
  --csv /data/LIVE/filelist.csv \
  --outdir images/custom_set \
  --json scripts/custom_2afc.json \
  -n 30 --seed 123
```

Dependencies: Requires Pillow (pip install Pillow).

3.2.1 createsuperuser — Create Admin Account

Command:

```
python manage.py createsuperuser
```

Interactive prompt for username, email, and password. The superuser has full access to the admin panel and the bulk-user-creation page.

3.2.2 runserver — Start Development Server

Command:

```
python manage.py runserver 0.0.0.0:8000
```

Bind to 0.0.0.0 to allow access from other machines on the network (useful for lab testing).

4 Researcher's Guide

This section describes the full workflow for a researcher setting up and running a subjective test.

4.1 Step 1: Installation

Ensure Python 3.11+ and Django are installed:

Command:

```
cd iqa_subjective_test
pip install -r requirements.txt
python manage.py makemigrations iqa
python manage.py migrate
python manage.py createsuperuser
```

4.2 Step 2: Prepare Images

Place all test images (and reference images, if applicable) into the *images/* directory. Subdirectories are allowed:

```
images/
├── references/
│   ├── scene_01.png
│   └── scene_02.png
├── method_a/
│   ├── scene_01_q30.png
│   └── scene_02_q30.png
└── method_b/
    ├── scene_01_q50.png
    └── scene_02_q50.png
```

Supported formats: any format the browser can display (PNG, JPEG, WebP, BMP, etc.).

4.3 Step 3: Create a Study

There are two ways to create a study.

4.3.1 Option A: JSON import (recommended for large studies)

Write a JSON file describing the study (see the `import_study` command reference above), then run:

Command:

```
python manage.py import_study my_study.json --activate
```

4.3.2 Option B: Django admin panel

1. Navigate to `http://localhost:8000/admin/`.
2. Under **IQA**, click **Images** → **Add Image** to register each image file.
3. Click **Studies** → **Add Study**.
4. Fill in the study name, mode, prompt, scale settings (for MOS), and sampler.
5. In the inline section at the bottom, add stimuli:
 - For MOS: set the test image and optional reference for each row.
 - For 2AFC: set image A, image B, and optional references for each row.

6. Set `is_active = True` when ready for subjects.
7. Click **Save**.

4.4 Step 4: Create Subject Accounts

4.4.1 Option A: Bulk creation via the web interface

Navigate to `http://localhost:8000/iqa/bulk-create-users/` (superuser login required). You can either:

- Enter specific usernames (one per line), or
- Specify a number and the system generates `user1`, `user2`, ... with random passwords.

The resulting credentials are displayed once. **Save them immediately**—passwords cannot be recovered.

4.4.2 Option B: Django admin

Navigate to `http://localhost:8000/admin/auth/user/` and add users manually.

4.5 Step 5: Run the Test

1. Start the server: `python manage.py runserver`
2. Distribute credentials to subjects.
3. Subjects navigate to `http://<host>:8000/iqa/login/`, log in, and begin evaluating.
4. The system automatically serves stimuli according to the selected sampler and tracks progress per user.

Subjects can close the browser and resume later—their progress is saved. If a subject revisits a previously-rated stimulus, a warning is shown and re-submitting overwrites the old response.

4.6 Step 6: Monitor and Export Results

4.6.1 Web viewer

Staff users (superuser or `is_staff=True`) can view responses at:

`http://localhost:8000/iqa/responses/`

Select a study from the dropdown to see a table of all responses.

4.6.2 CSV export (web)

From the response viewer page, click the **Export CSV** button. The downloaded CSV contains:

For MOS studies:

user	stimulus_order	image	reference	score
------	----------------	-------	-----------	-------

For 2AFC studies:

user	stimulus_order	image_a	image_b	reference_a	reference_b	choice
------	----------------	---------	---------	-------------	-------------	--------

The CSV can also be downloaded directly:

Command:

```
curl http://localhost:8000/iqa/responses/export/<study_id>/
```

4.6.3 CSV export (command line)

The `export_responses` management command provides a richer CSV that also includes the study name and timestamp columns. See the full command reference in [Section 3.1.3](#).

Command:

```
python manage.py export_responses --study 3 -o results.csv
```

4.7 Study Configuration Reference

Field	Default	Description
name	—	Display name shown to subjects on the home page.
mode	—	MOS or 2AFC. Determines the evaluation interface and response type.
prompt	empty	Instruction text shown above the images during evaluation.
scale_min	1	MOS only. Lowest score on the rating scale.
scale_max	5	MOS only. Highest score on the rating scale.
scale_min_label	Bad	MOS only. Label displayed below the lowest score button.
scale_max_label	Excellent	MOS only. Label displayed below the highest score button.
sampler	sequential	One of <code>sequential</code> , <code>random</code> , <code>least_evaluated</code> .
image_sizing	fit_screen	How images are sized in the evaluation UI. One of <code>fit_screen</code> , <code>original</code> , <code>scaled</code> .
image_scale_factor	1.0	Scale factor applied when <code>image_sizing</code> is <code>scaled</code> . For example, 0.5 displays images at half their native resolution.
zoom_enabled	False	When enabled, hovering over any image shows a magnified overlay on every image on the page, letting subjects inspect the same region across reference and test images.
zoom_factor	2.0	Magnification factor for the zoom overlay. Only used when <code>zoom_enabled</code> is <code>True</code> .
is_active	False	Only active studies are visible to subjects.

4.7.1 Image Sizing Modes

- **fit_screen** (default) — All images (reference and test) are constrained to fit within the browser viewport. Good for varying screen sizes and large images.
- **original** — Images are displayed at their native pixel resolution. If images are larger than the viewport, the panel scrolls.
- **scaled** — Images are displayed at `native_size × image_scale_factor`. For example, a 1024×768 image with factor 0.5 is shown as 512×384 pixels. Both reference and test images are scaled by the same factor.

4.7.2 Zoom Overlay

When `zoom_enabled` is `True`, hovering the mouse over any image during evaluation displays a magnified overlay on **every** image on the page simultaneously. This allows subjects to inspect the same spatial region across reference and test images at once.

- The overlay is 1/3 of the image width by 1/3 of the image height.
- The magnification level is controlled by `zoom_factor` (e.g., `2.0` for $2\times$ zoom, `4.0` for $4\times$).
- The overlay is positioned at the corner of each image that is farthest from the mouse cursor, so it never obscures the area being inspected.
- Moving the mouse pans the zoomed view; moving off the image hides all overlays.

5 Subject's Guide

This section is intended to be shared with test participants.

5.1 Logging In

1. Open the test URL provided by the researcher in your browser (e.g., `http://192.168.1.10:8000/iqa/login/`).
2. Enter the **username** and **password** you were given.
3. Click **Log In**.

5.2 Home Page

After logging in, you will see a list of available studies. Each study card shows:

- The study name.
- The evaluation type (Mean Opinion Score or Two-Alternative Forced Choice).
- An optional instruction text.

Click **Start Evaluation** on the study you wish to participate in.

5.3 MOS Evaluation (Single Image Rating)

You will see one or two images on the screen:

- If a **reference image** is provided, it appears on the **left** side, labelled “Reference”. The image you are rating appears on the **right**, labelled “Test Image”.
- If there is no reference, only the test image is shown.

Below the images is a **rating scale** (typically 1 to 5). The endpoints are labelled (e.g., “Bad” and “Excellent”).

1. Look at the image(s) carefully.
2. Click the score button that best reflects your quality assessment.
3. Click **Submit**.

The system automatically advances to the next image.

A progress counter (e.g., “3 / 20”) is shown in the top-right corner so you know how many images remain.

Keyboard shortcuts (button-based scoring with values 1–9 only):

- Press a **number key** (1–9) to select the corresponding score.
- Press **Enter** to submit.

When using the text-box input variant, you can type a score directly and press **Enter** to submit.

5.4 2AFC Evaluation (Pair Comparison)

You will see two images displayed side by side, labelled **Image A** (left) and **Image B** (right).

- If reference images are provided, they appear above their corresponding test images.
- If both images share the same reference, it is shown once at the top.

1. Compare the two images.
2. Click **Choose A** or **Choose B** to indicate which image you think has better quality.
3. Click **Submit**.

The system automatically advances to the next pair.

Keyboard shortcuts:

- Press **A** or **1** to select Image A.
- Press **B** or **2** to select Image B.
- Press **Enter** to submit.

5.5 General Tips

- **Take your time.** There is no time limit per image.
- **Neutral background.** Images are displayed on a neutral grey background to avoid colour bias—this is intentional.
- **Zoom overlay.** Some studies enable a zoom feature. When available, hover your mouse over any image to see a magnified inset on each image. The inset tracks your cursor position, so you can compare fine details across reference and test images at the same time. Moving the mouse away from the image hides the overlay.
- **You can take breaks.** Close the browser at any time; your progress is saved. When you return and log in again, click “Start Evaluation” and you will continue from where you left off.
- **Changing your mind.** If you return to a previously-rated stimulus, a warning appears. You can submit a new response and it will replace the old one.
- **Completion.** When all stimuli in a study have been rated, you will see a “All Done!” message. You can then return to the home page.

5.6 Logging Out

Click the **Logout** button on the home page when you are finished.

5.7 Additional Instructions for Subject Running the Server on their Own Machine

To start the server, open a terminal, navigate to the `iqa_subjective_test/` directory, and run:

```
python manage.py runserver
```

After finishing the evaluation, please export your responses to a CSV file. Either by clicking on the “Export” button or using the following command:

```
python manage.py export_responses -s <study-name> -o res.csv
```

For the first internal test, study name is “ADP_train_2000 (2AFC)”

6 URL Reference

All URLs are prefixed with /iqa/.

URL Pattern	Method	Description
/iqa/	GET	Redirects to /iqa/home/.
/iqa/home/	GET	Lists active studies. Requires login.
/iqa/login/	GET/POST	Login page.
/iqa/logout/	POST	Logs the user out.
/iqa/next-stimulus/	POST	Picks the next stimulus via sampler; redirects to the evaluation page.
/iqa/evaluate/mos/<study_id>/<stimulus_id>/	GET	MOS evaluation page for a specific stimulus.
/iqa/evaluate/pair/<study_id>/<stimulus_id>/	GET	2AFC evaluation page for a specific stimulus.
/iqa/submit/	POST	Receives the evaluation form submission.
/iqa/bulk-create-users/	GET/POST	Bulk user creation (superuser only).
/iqa/user-creation-results/	GET	Shows credentials of just-created users.
/iqa/responses/	GET	Response viewer with study filter (staff only).
/iqa/responses/export/<study_id>/	GET	CSV download of all responses for a study (staff only).